

Guia de Estudo — Solução Big Data para Frota Autônoma Conectada

Avaliação 03 (final) · MAE016.15912 / EEL890 — TED-B: Big Data & Data Warehouse · UFRJ/NCE · Prof. Milton Ramos Ramirez · 2026.1 Grupo: Izabela Lima da Silva (DRE 124156557) · Caio Meirelles (DRE 122071557)

Material de estudo escrito para dominar o projeto **de dentro para fora**: fundamentos das quatro partes da ementa, cada decisão de arquitetura com o *porquê* e a referência que a sustenta, o modelo de DW e sua extensão, glossário, cheat-sheet das 21 referências e flashcards para o Q&A. Estude na ordem: **§1 (mapa mental)** → **§2–5 (fundamentos)** → **§6 (arquitetura)** → **§10 (flashcards)**.

0. Como usar este guia e o formato da defesa

A avaliação é uma **defesa oral de ~1 hora** (≈45 min de apresentação + ≈15 min de perguntas). O critério central do professor é **explicar e justificar** cada escolha — não basta citar a tecnologia, é preciso dizer *por que* ela, *que problema* resolve, *qual referência* a fundamenta e *que alternativa* foi descartada. Este guia foi montado para você conseguir responder a qualquer “por quê?”.

Três níveis de resposta que você deve treinar para cada tópico: 1. **Uma frase** (o que é / o que resolve) — para fluir na apresentação. 2. **Um parágrafo** (o mecanismo + o trade-off) — para uma pergunta direta. 3. **A referência** (quem provou isso e qual a ideia-chave) — para impressionar.

1. O mapa mental (a visão em uma página)

A tese central do trabalho: *o esquema estrela em PostgreSQL da Avaliação 02 deixa de ser um Data Warehouse isolado e passa a ser a camada **Gold** de um **Lakehouse** orientado a fluxo contínuo*. Ou seja: não jogamos fora o DW — nós o **envolvemos** numa arquitetura de Big Data em tempo real.

O problema em uma frase: um consórcio de 6 locadoras opera uma frota de **veículos autônomos e conectados**; cada veículo é um “sensor sobre rodas” que gera dados sem parar, sob **banda móvel cara e finita**, e o sistema precisa monitorar a frota, cobrar automaticamente, responder a emergências e oferecer um concierge por IA — tudo em tempo real, sem perder pacotes e com ordem estrita por veículo.

O pipeline (decore este fluxo):

Veículo (Edge/Avro) → MQTT → Kafka → [Flink | Spark] → Delta Lake (Bronze→Silver→Gold)
→ NoSQL (Cassandra/Mongo/Redis)
→ Serving (Streamlit + Cobrança + Concierge)

Os três “caminhos” de dados (thermal paths): - **Hot path** (milissegundos–segundos): colisão detectada na borda → alerta → resposta a emergência. - **Warm path** (segundos): telemetria agregada a cada 1 s → análise contínua, rotas, score de condução. - **Cold path** (minutos–horas): histórico → relatórios, manutenção preditiva, matriz de Markov.

A frase de elevador: “*Filtramos na borda para poupar banda, isolamos os fluxos no Kafka com ordem por veículo, adotamos a arquitetura Kappa realizada por um Lakehouse Delta (um só código, ACID entre lote e streaming), processamos com Spark e Flink conforme o SLA, persistimos de forma poliglota segundo o CAP, e servimos dashboard, cobrança auditável, dossiês de sinistro e um concierge com RAG — tudo fundamentado na literatura e entregando valor mensurável ao consórcio.*”

2. Parte I — Fundamentos: VLDB e computação distribuída

2.1 Por que “muito volumoso” (VLDB) muda tudo

Quando os dados não cabem numa máquina, você precisa de um **cluster** de máquinas baratas (*commodity hardware*). E aí surge o princípio que reorganiza todo o projeto: **em um cluster grande, falhas de componentes são a norma, não a exceção** — discos, redes e nós falham o tempo todo. Logo, **monitoramento, detecção e recuperação automática** têm de ser parte do desenho, não um remendo. (*Ghemawat et al., 2003 — Google File System.*) - **No nosso projeto:** assumimos a falha como estado normal do consórcio → réplicas no Kafka e no Cassandra, o Delta como fonte da verdade recuperável, e containers reconstituíveis via docker-compose.

2.2 Localidade: mova a computação, não os dados

Largura de banda de rede é o recurso escasso. Em vez de trazer terabytes até o processador, leve o código até onde o dado está. (*Dean & Ghemawat, 2004 — MapReduce.*) - **No nosso projeto:** isso fundamenta a **computação de borda (Edge)** — o veículo pré-agrega a telemetria antes de transmitir, poupando a banda móvel cara. É o *combiner* do MapReduce aplicado ao veículo.

2.3 Escalar na horizontal e compor por camadas

Não se cresce comprando um servidor gigante (vertical); cresce-se **adicionando máquinas** (horizontal). E sistemas grandes reusam infraestrutura em camadas especializadas — o Bigtable foi construído sobre o GFS e coordenado por consenso (Paxos/Chubby). (*Chang et al., 2006.*) - **No nosso projeto:** o pipeline broker → Kafka → processamento → persistência é essa composição em camadas, cada uma fazendo uma coisa bem.

2.4 Tempo e ordem em sistemas distribuídos

Num sistema distribuído, “que evento aconteceu primeiro?” é uma pergunta difícil — não há um relógio global perfeito. O **Spanner** resolve expondo a incerteza do relógio como um intervalo [earliest, latest] (TrueTime) e garantindo **consistência externa**: a ordem de commit reflete a ordem real dos fatos. (*Corbett et al., 2012.*) - **No nosso projeto:** é o alicerce teórico da exigência de “**ordenação estrita por veículo e timestamp**”. O Flink obtém o mesmo efeito prático com *event-time* e *watermarks*.

2.5 Meça a cauda, não a média

A qualidade percebida de um serviço é definida pelos piores casos. Por isso o SLA se mede no **percentil 99,9**, não na média. (*DeCandia et al., 2007 — Dynamo.*) - **No nosso projeto:** vale sobretudo para o apoio a emergências — o que importa é o pior tempo de resposta, não o médio.

3. Parte II — Data Warehouse e a nova era do BI

3.1 OLTP vs OLAP (o axioma do DW)

- **OLTP** (Online Transaction Processing) = o banco operacional do dia a dia: muitas transações pequenas (inserir uma reserva, atualizar um pagamento). Otimizado para **escrita** e consistência.
- **OLAP** (Online Analytical Processing) = análise: poucas consultas enormes, com varreduras e agregações (quantos carros por pátio no último mês?). Otimizado para **leitura analítica**.
- **A regra de ouro:** o DW é uma coleção *subject-oriented, integrated, time-varying, non-volatile*, **fisicamente separada** dos bancos operacionais — porque rodar consultas OLAP em cima do OLTP degradaria as transações. O paper cita explicitamente *fleet management* como caso clássico. (*Chaudhuri & Dayal, 1997.*)
- **No nosso projeto:** o dashboard roda **sobre a camada Gold**, nunca sobre os OLTP de reserva/cobrança.

3.2 Modelagem dimensional (o coração da Aval. 02)

- **Tabela-fato:** guarda as **medidas** (números que se somam/agregam) e chaves para as dimensões. Ex.: Fato_Locacao com qtd_locacoes, dias_locacao.
- **Dimensão:** o contexto descritivo pelo qual você fatia/agrupa. Ex.: Dim_Veiculo (categoria, marca), Dim_Tempo (dia, mês, ano).
- **Star schema (esquema estrela):** um fato no centro cercado de dimensões. **Snowflake:** dimensões normalizadas em sub-tabelas (mais junções, menos redundância).
- **Surrogate key (chave substituta):** um inteiro artificial (sk_veiculo) como PK da dimensão, em vez da chave do sistema de origem — desacopla o DW das fontes e acelera as junções.
- **Dimensões conformadas:** a *mesma* dimensão significa a mesma coisa em vários fatos (ex.: Dim_Tempo é idêntica para Reserva, Locação e Movimentação) → permite relatórios integrados entre as 6 locadoras.
- **Fact constellation (constelação de fatos):** vários fatos compartilhando dimensões conformadas — é exatamente o nosso caso (3 fatos, 5 dimensões).
- **Role-playing (papel múltiplo):** uma dimensão física servindo em vários papéis. Nossa Dim_Patio é pátio de retirada, devolução, origem e destino — via 4 views.
- **SCD (Slowly Changing Dimension):** como tratar mudanças no cadastro. **Tipo 1** = sobrescreve (perde histórico); **Tipo 2** = versiona (nova linha com vigência, guarda histórico). Nosso DW usa Tipo 1; na extensão, Dim_Sensor é **Tipo 2** (para reconstruir o dossiê “como era na hora do sinistro”).

3.3 OLAP e o operador CUBE

Operações analíticas: **roll-up** (subir na hierarquia: veículo→pátio→empresa→frota), **drill-down** (descer), **slice-and-dice** (filtrar/pivotar). A generalização N-dimensional é o operador **CUBE** com o valor especial ALL, que pré-computa todos os subtotais. (Gray et al., 1997.) - **Detalhe fino que impressiona:** agregados **distributivos** (COUNT, SUM) e **algébricos** (AVG = soma+contagem) computam-se incrementalmente e cabem em janelas de streaming com estado pequeno; agregados **holísticos** (mediana, percentis) exigem ver todos os dados e são adiados ao batch. Isso justifica *o que fica no hot path* e *o que fica no batch*.

3.4 ETL → ELT (a modernização)

- **ETL clássico:** *Extract → Transform → Load* — transforma **antes** de carregar (transformação num servidor separado).
- **ELT moderno:** *Extract → Load → Transform* — carrega o **bruto** no lago/Lakehouse e transforma **lá dentro**, usando o poder do cluster (Spark). É o que a ementa pede.
- **Orquestração:** quem dispara e coordena as etapas do pipeline em ordem, com retries e agendamento — **Airflow / Prefect / Dagster** (DAGs de tarefas). No nosso full, um DAG do Airflow orquestra o ELT Silver→Gold.

3.5 Otimização e escala (MPP)

- **MPP (Massively Parallel Processing):** dividir a tabela em muitas máquinas e processar em paralelo.
- **Particionamento** (por data/veículo), **indexação** (bitmap/join index), **compressão colunar** — reduzem I/O.
- **Cloud DWs:** Snowflake, BigQuery, Redshift, Synapse — DWs gerenciados que separam **armazenamento e computação**.

3.6 O Data Lakehouse (o clímax da Parte II)

A arquitetura tradicional tem **dois níveis:** um *data lake* barato (arquivos) + um *data warehouse* caro. Problemas: dados ficam **velhos** (staleness — 86% dos analistas usam dados desatualizados), suporte fraco a ML, e alto custo total. O **Lakehouse** unifica os dois: um só lugar com governança de DW sobre arquivos de lago. (Armbrust et al., 2021.) - **Como funciona:** um object store (S3/MinIO) é *key-value* e **não tem transações**. O **Delta Lake** adiciona um **log de transações** por cima, dando **ACID**, *time travel* (versões passadas), *data skipping* (pular arquivos via estatísticas min/max) e MERGE/DELETE. (Armbrust et al., 2020.) - **Medallion:** **Bronze** (bruto, append-only) → **Silver** (limpo, deduplicado, conformado) → **Gold** (marts de negócio = o nosso DW estrela estendido). - **O motor:** **Spark SQL**, com o otimizador **Catalyst** (análise → otimização lógica → plano físico por custo → geração de código). (Armbrust et al., 2015.)

4. Parte III — Big Data, streaming e engenharia de IA

4.1 O que é Big Data: os “V’s” e a ética

- **Volume** (quantidade), **Velocidade** (taxa de chegada), **Variedade** (estruturado/semi/não), + **Veracidade** (qualidade) e **Valor** (o fim). Frota conectada = os 3 V’s clássicos ao extremo.
- **Ética/privacidade**: dados de localização e de condução são sensíveis → consentimento, minimização, LGPD.

4.2 Armazenamento escalável: HDFS e cloud stores

O GFS/HDFS dá semântica **append-only** (só acrescenta) com *record append* atômico e integridade por **checksum**. (Ghemawat et al., 2003.) Cloud stores: **S3, ADLS, GCS** (e MinIO, que é S3 self-hosted). - **Ideia-chave**: telemetria é uma **série temporal só de acréscimo** — o mesmo modelo do log do Kafka e do transaction log do Delta.

4.3 MapReduce (o paradigma batch)

Restringir o programa a duas funções — **map** (transforma cada registro) e **reduce** (agrega por chave) — é o que permite ao runtime **paralelizar, escalonar e tolerar falhas automaticamente**. A tolerância vem da **re-execução determinística** (refez a tarefa perdida). (Dean & Ghemawat, 2004.) - **Combiner**: agregação parcial no lado do produtor (poupa rede) → fundamenta a pré-agregação na borda. - **Backup tasks**: contra *stragglers* (tarefas lentas), roda cópias especulativas.

4.4 Spark e o RDD (por que superou o Hadoop)

O Hadoop MapReduce grava em disco entre cada job (lento). O Spark introduz o **RDD (Resilient Distributed Dataset)**: uma coleção particionada e **imutável** em memória, cuja tolerância a falhas vem da **linhagem (lineage)** — se uma partição se perde, recomputa-se a partir dos pais, **sem replicar**. Reuso em memória dá até **20x** em algoritmos iterativos. (Zaharia et al., 2012.) - **DAG de estágios**: a avaliação é preguiçosa; dependências *narrow* (pipelined, sem embaralhar) vs *wide* (shuffle). Fixar o particionador por `vehicle_id` **co-localiza** as junções e evita shuffle. - **No nosso projeto**: o Spark faz o batch (km, Markov, manutenção) e o ETL Bronze→Silver→Gold.

4.5 Kafka (o coração da ingestão em tempo real)

Kafka é um **log distribuído, particionado e durável**. (Kreps et al., 2011.) Conceitos: - **Tópico**: um canal nomeado (ex.: `telemetry.raw`). - **Partição**: o tópico é dividido em partições; dentro de uma partição a **ordem é estrita**. Particionar por `vehicle_id` coloca toda a telemetria de um veículo na **mesma partição** → ordem por veículo **sem coordenação global**. - **Offset**: a posição do consumidor no log; o consumo é **pull** e permite **replay** (reprocessar do zero). - **Retenção**: o broker guarda por tempo, funcionando como **buffer durável** sob banda finita. - **Schema Registry + Avro**: contrato de esquema entre a borda e os consumidores, com evolução segura. - **Entrega at-least-once**: pode duplicar → a **deduplicação** (por `vehicle_id+timestamp`) fica na aplicação, evitando o custo de commit em duas fases.

4.6 Processamento de stream: os conceitos que a banca cobra

- **Stream** = fluxo **ilimitado** de eventos que chegam **fora de ordem** (múltiplas fontes, rede instável).
- **Event-time vs processing-time**: processar pelo horário em que o evento **aconteceu** (carimbado na borda), não em que **chegou** ao servidor.
- **Watermark**: uma marca d’água que diz “já vi todos os eventos até o tempo T”; define quanto atraso tolerar antes de fechar uma janela. É como o Flink reordena pacotes atrasados. (Fragkoulis et al., 2020.)
- **Janela (window)**: agrupar eventos por tempo (tumbling/sliding) para agregar.
- **Estado (state)**: o operador guarda memória entre eventos (ex.: velocidade média por veículo).
- **Exactly-once**: cada evento afeta o resultado exatamente uma vez, mesmo com falhas.

4.7 Spark Streaming vs Flink

- **Spark Streaming (D-Streams)**: trata o stream como **micro-batches** determinísticos; recupera por linhagem e garante **exactly-once** com saídas idempotentes. Ótimo quando você quer o **mesmo código** de batch e stream +

MLlib. (Zaharia et al., 2013.) → **nosso hot path de emergências**.

- **Flink**: um **dataflow único** onde batch é caso especial de stream; **event-time + watermarks** nativos, janelamento flexível, **estado gerenciado** por chave, e **exactly-once** via *Asynchronous Barrier Snapshotting* (ABS) — snapshots consistentes **sem parar** a execução (estende o algoritmo de Chandy-Lamport). Latência de milissegundos. (Carbone et al., 2015.) → **nosssa análise contínua** (rotas, score).

4.8 Lambda vs Kappa (a decisão-chave)

- **Lambda**: duas camadas — **batch** (exata, lenta) + **speed** (rápida, aproximada) — e um serving que junta as duas. Problema: **duas bases de código** com a mesma lógica → manutenção dobrada e risco de divergência.
- **Kappa**: **tudo é stream**; para reprocessar o histórico, você **reproduz o log** (replay). Uma só lógica.
- **Nossa decisão**: o problema é *streaming-first*; o stream durável do Kafka unifica os caminhos e **elimina a dupla implementação**. Adotamos **Kappa**, e cobrimos o lote pesado com o **Lakehouse** (Delta) — que dá o reprocessamento em batch sem um segundo código. (Fragkoulis 2020; Carbone 2015; Armbrust 2020.)

4.9 Engenharia de IA: RAG, agentes, MLOps

- **RAG (Retrieval-Augmented Generation)**: o modelo generativo consulta uma **base de conhecimento recuperável** (índice de embeddings) antes de responder. Vantagens: **reduz alucinação**, dá **proveniência** (você sabe de onde veio a resposta) e permite **trocar o índice sem re-treinar**. (Lewis et al., 2020.)
- **No nosso projeto**: o **concierge** usa PLN + RAG local (sem LLM paga) para montar roteiros a partir de POIs; e os **dossiês de emergência** são “citáveis” pela mesma lógica de recuperação.
- **MLOps**: operacionalizar modelos de ML como pipelines (treino, versionamento, deploy, monitoramento) — no full, o score de condução e a manutenção preditiva.

5. Parte IV — NoSQL, CAP, NewSQL e bancos vetoriais

5.1 O teorema CAP (a raiz da persistência poliglota)

Sob uma **partição de rede** (P), você não pode ter ao mesmo tempo **Consistência** (C) e **Disponibilidade** (A) — tem de escolher. (Gilbert & Lynch, 2002.) A saída prática: **segmentar** o sistema e dar a cada subserviço o seu trade-off. - **CP** (consistência forte, pode ficar indisponível): cobrança, dossiê. - **AP** (sempre disponível, consistência eventual): telemetria. - Analogia clássica: o **carrinho de compras** (AP — sempre aceita adicionar) vs o **checkout** (CP — o pagamento tem de ser exato).

5.2 ACID vs BASE

- **ACID** (bancos relacionais): Atomicidade, Consistência, Isolamento, Durabilidade — garantias fortes.
- **BASE** (muitos NoSQL): *Basically Available, Soft state, Eventual consistency* — troca garantias por escala e disponibilidade.

5.3 Os quatro modelos NoSQL (Cattell, 2011)

- **Key-value** (Redis, DynamoDB): mapa chave→valor, rapidíssimo, em memória. → **cache** ao vivo, rotas, geoespacial.
- **Documento** (MongoDB, Firestore): JSON flexível. → **cadastrais** e **dossiês** de sinistro heterogêneos.
- **Wide-column** (Cassandra, Bigtable): mapa esparsa e ordenado, escrita massiva. → **telemetria**.
- **Grafo** (Neo4j): nós e arestas, ótimo para relacionamentos/roteirização e raciocínio de agente.

5.4 Dynamo e Cassandra (o coração da telemetria)

- **Dynamo** (DeCandia et al., 2007): loja **always-writeable** (nunca recusa escrita), com **consistent hashing** (distribui chaves em um anel com nós virtuais), **quóruns ajustáveis N/R/W** (você sintoniza consistência vs

latência), **sloppy quorum** / **hinted handoff** (aceita escrever mesmo com nós fora), **árvores de Merkle** (anti-entropia) e **vector clocks** (causalidade). É o que garante “sem perda de pacotes” com conectividade intermitente.

- **Cassandra** (*Lakshman & Malik, 2009*): casa Dynamo (distribuição) + Bigtable (modelo de dados). Escrita altíssima pelo motor **LSM** (*Log-Structured Merge*: commit log + memtable em RAM + SSTable imutável em disco + compaction). O **clustering por timestamp** dentro da partição `vehicle_id` materializa fisicamente a **ordenação estrita**.

5.5 NewSQL e bancos vetoriais (a fronteira)

- **NewSQL**: a busca por **SQL + ACID que escala horizontalmente** (ex.: Spanner). Mostra que consistência forte não precisa ser sacrificada para escala em operações de escopo único → posiciona o **PostgreSQL** como o lugar legítimo da cobrança e da modelagem dimensional.
- **Bancos vetoriais** (Pinecone, Milvus, Weaviate, pgvector): guardam **embeddings** (vetores que representam significado) e fazem **busca por similaridade** (ANN via HNSW). São a “**memória**” de **LLMs e agentes**. (*Guo et al., 2022 — Manu.*) → base do RAG do concierge.

6. A arquitetura do projeto, camada por camada

Para cada camada, saiba responder: **o que faz · qual tecnologia · por quê (referência) · alternativa descartada · valor ao cliente**.

Camada	Tecnologia	Por quê (fundamento)	Descartamos	Valor
Borda	Edge + Avro/MQTT	Localidade/combiner poupa banda (<i>Dean 2004</i>)	enviar vídeo bruto	↓ custo de banda
Ingestão	Apache Kafka	Log particionado, ordem por veículo, replay (<i>Kreps 2011</i>)	HTTP síncrono no banco	sem perda, sem gargalo
Conectividade	Kafka + Flink watermarks	Ordem sob rede oscilante (<i>Corbett 2012; Carbone 2015</i>)	ordenar na aplicação	dados confiáveis
Arquitetura	Kappa + Delta Lake	Um código; ACID lote↔stream (<i>Fragkoulis 2020; Armbrust 2020</i>)	Lambda (2 códigos)	menos manutenção
Processamento	Spark + Flink	3 SLAs: batch/emergência/contínuo (<i>Zaharia 2012/13; Carbone 2015</i>)	um motor só	latência certa por tarefa
Lakehouse	Delta/MinIO (Bronze/Silver/Gold)	ACID, time travel no object store (<i>Armbrust 2020/21</i>)	lake + DW separados	uma verdade, auditável
Persistência	Cassandra/Mongo/Redis/PG	CAP: cada dado no banco certo (<i>Gilbert 2002; Lakshman 2009</i>)	banco único	escala + latência
Serving	Streamlit + Redis	OLAP sobre a Gold, ao vivo (<i>Chaudhuri 1997; Gray 1997</i>)	dashboard sobre OLTP	decisão em tempo real
Cobrança	Spark Streaming (event-sourcing)	Exactly-once idempotente (<i>Zaharia 2013</i>)	recalcular do zero	↑ receita, auditável
Emergências	hot path + dossiê	detecção na borda, cauda de latência (<i>DeCandia 2007</i>)	processar tudo na nuvem	↓ tempo de resposta
Concierge	PLN + RAG + vetorial	proveniência, sem alucinação (<i>Lewis 2020;</i>	LLM sem recuperação	↑ experiência/NPS

7. O modelo de DW (Aval. 02) e a extensão (Aval. 03)

7.1 O que fizemos na Avaliação 02 (o DW base)

- **Esquema estrela** em PostgreSQL (schema dw_locadora), integrando **4 fontes OLTP** via postgres_fdw numa **staging** neutra.
- **3 fatos** (fact constellation): Fato_Reserva, Fato_Locacao, Fato_Movimentacao — todos com medida COUNT (agregados).
- **5 dimensões conformadas**: Dim_Tempo (grão dia), Dim_Cliente, Dim_Veiculo, Dim_Empresa, Dim_Patio (role-playing: retirada/devolução/origem/destino via 4 views).
- **ETL em 5 scripts** (DDL → extração/staging → transformação/conformação → carga full-refresh → relatórios).
- **Matriz de Markov**: a partir de Fato_Movimentacao, $P(\text{pátio } i \rightarrow \text{pátio } j) = \text{mov}(i,j) / \sum \text{mov}(i,\cdot)$. É uma **cadeia de Markov de 1ª ordem** (estimada por contagem de frequências) que prevê a **ocupação dos pátios** e apoia a **redistribuição** da frota.

7.2 O que estendemos na Avaliação 03 (o DW vira Gold)

Lacuna do DW base: é **batch (D-1)**, grão de dia, **sem um fato financeiro materializado** (o Pagamento/valor existia só no OLTP da Parte I), sem telemetria. A extensão (arquivo 06_ddl_extensao.sql, validado num PostgreSQL real) adiciona: - **4 fatos novos**: Fato_Telemetria_Diaria (snapshot veículoxdia: km, velocidade, consumo, eventos bruscos), Fato_Cobranca (traz o Pagamento do OLTP para a Gold e o enriquece; medidas aditivas base → valor_final), Fato_Sinistro (severidade, custo, tempo de resposta), Fato_Manutencao (custo, downtime, probabilidade de falha). - **4 dimensões novas**: Dim_Tempo_Detalhe (grão minuto), Dim_Sensor (**SCD-2** com firmware versionado, para dossiê point-in-time), Dim_TipoEvento, Dim_FaixaConducao (banda do score, ligada à tarifa). - **Carga por ELT do Silver** (agregados do Flink/Spark), não mais só por batch OLTP. - **Markov estendida**: de recomputação batch para **condicional** (por faixa horária/categoria) e **incremental** (janela deslizante) — servindo o reposicionamento do veículo autônomo vazio em tempo quase real.

7.3 Três casos resolvidos (a modelagem em ação)

Estude estes três cenários — eles mostram **onde cada entidade vive** e por que a extensão foi necessária. Na defesa, se o professor perguntar “onde entra a tabela X?”, aponte o caso.

Caso 1 — Colisão em túnel (crítico / resposta rápida). - *Problema*: às 18h23 o veículo autônomo colide **dentro de um túnel** (zona de sombra, sem sinal). - *Resposta, passo a passo*: (1) o sensor de impacto detecta a desaceleração violenta **na borda** em sub-segundo; (2) dispara um payload de emergência (ring buffer de ~30 s) em alerts.critical, priorizando a rede; (3) a telemetria pré-impacto retida em *store-and-forward* só sai do túnel depois e chega **fora de ordem** — o Flink a reordena por *event-time/watermark*; (4) o Spark Streaming correlaciona e orquestra guincho, seguradora e a reserva de um substituto; (5) monta-se o **dossiê** com o *time travel* do Delta + a Dim_Sensor (SCD-2), sabendo qual firmware estava ativo no minuto exato. - *Entidades que acendem*: Fato_Sinistro, Dim_TipoEvento, Dim_Sensor, Dim_Tempo_Detalhe, Dim_Veiculo, Dim_Patio, Dim_Cliente. - *Por que aponta modificação*: a Dim_Tempo (grão dia) não situa o evento no minuto → justifica a Dim_Tempo_Detalhe; sem Dim_Sensor SCD-2 não há dossiê *point-in-time*. - *Valor*: ↓ tempo de resposta e de regulação.

Caso 2 — Devolução com condução agressiva (usual / financeiro). - *Problema*: o cliente devolve o veículo após 3 dias com excesso de velocidade recorrente e uma multa. - *Resposta*: em *warm path*, o Flink computou continuamente o score de condução (janelas + estado por veículo) → Dim_FaixaConducao = “Agressivo” (com fator_tarifa). Na devolução, o evento em billing.events aciona o motor de cobrança (*exactly-once* idempotente), que consolida a Fato_Telemetria_Diaria (km, velocidade, eventos bruscos) + a infração e calcula a Fato_Cobranca = base ± acréscimos. - *Entidades*: Fato_Cobranca, Fato_Telemetria_Diaria, Fato_Locacao, Dim_FaixaConducao, Dim_TipoEvento, Dim_Cliente, Dim_Veiculo, Dim_Tempo. - *Por que aponta modificação*: o esquema estrela da Aval.

02 não materializou um fato financeiro (o Pagamento/valor existia só no OLTP da Parte I) → a Fato_Cobrança traz o pagamento para a Gold e o **enriquece** com os ajustes dinâmicos (km/consumo/infração/score); o score derivado vira a Dim_FaixaConducao. - *Valor*: ↑ receita por locação; ↓ contestações (auditável por *time travel*).

Caso 3 — Pico no aeroporto, pane iminente e concierge (analítico / IA). - *Problema*: sexta 17h, a demanda concentra-se no Galeão; a frota precisa se reposicionar; um veículo mostra sinais de falha; e um cliente premium pede um roteiro. - *Resposta*: a **matriz de Markov condicional** (por faixa horária, via Dim_Tempo_Detalhe) prevê a migração e dispara o **reposicionamento** de veículos vazios (Fato_Movimentacao + Dim_Patio origem/destino); o batch Spark sobre a Fato_Telemetria_Diaria calcula a Fato_Manutencao (probabilidade de falha) → agenda manutenção e tira o veículo do *pool* (Redis); o concierge (PLN + RAG sobre banco vetorial) monta o roteiro e injeta *waypoints* no GPS. - *Entidades*: Fato_Movimentacao, Fato_Manutencao, Fato_Telemetria_Diaria, Dim_Patio, Dim_Tempo_Detalhe, Dim_Veiculo, Dim_Empresa. - *Por que aponta modificação*: a Markov incondicional torna-se condicional (exige Dim_Tempo_Detalhe); a manutenção preditiva exige os novos Fato_Manutencao e Fato_Telemetria_Diaria. - *Valor*: ↑ utilização; ↓ downtime; ↑ NPS.

Matriz-resumo — memorize as + (extensão):

Entidade	1 · Colisão	2 · Cobrança	3 · Frota
Fato_Sinistro +	dossiê + resposta	—	—
Fato_Cobrança +	recálculo	valor ± acréscimos	—
Fato_Telemetria_Diaria +	buffer pré-impacto	km, velocidade	base do score/manutenção
Fato_Manutencao +	—	—	prob. falha → agenda
Fato_Movimentacao	despacho	—	Markov → reposicionar
Dim_Sensor + (SCD-2)	firmware na hora	—	saúde do sensor
Dim_TipoEvento +	colisão	violação	pane
Dim_FaixaConducao +	—	Agressivo → tarifa	perfil
Dim_Tempo_Detalhe +	minuto 18:23	janelas	faixa horária/pico

Dica de estudo: para *cada* entidade da coluna, saiba dizer **um caso** em que ela aparece. É a pergunta mais provável sobre modelagem.

7.4 Casos limítrofes (como o sistema não quebra)

O enunciado exige **banda finita, sem perda de pacotes e ordenação estrita sob estresse**. A robustez vem de quatro garantias combinadas — **durabilidade, ordem, idempotência e reconciliação**:

Situação-limite	Como o sistema lida
Túnel / queda de conexão	<i>store-and-forward</i> na borda + <i>ring buffer</i> ; drena em ordem ao reconectar — zero perda
Dados fora de ordem / atrasados	ordem por <i>vehicle_id</i> no Kafka + <i>event-time</i> /watermarks no Flink; muito tardios reconciliam no lote
Relógio dessincronizado (<i>clock skew</i>)	carimbo na borda + <i>event-time</i> ; watermarks toleram o desvio
Duplicatas / <i>replay</i>	idempotência por chave; cobrança <i>exactly-once</i> (Delta) — nunca em dobro
Pico de carga / surto	Kafka como <i>buffer</i> elástico; <i>backpressure</i> ; tópico crítico isolado da telemetria
Falha de nó / <i>broker</i>	replicação/ISR (Kafka); recomputação por linhagem (RDD); ACID (Delta)
Sensor / firmware inválido	Schema Registry + <i>dead-letter</i> ; Dim_Sensor SCD-2 sabe o firmware da hora
Evento tardio pós-fatura	<i>time travel</i> + janela de conciliação — ajuste auditável, sem reescrever histórico

Situação-limite

Partição de rede (CAP)

Como o sistema lida

telemetria **AP** segue aceitando escrita; cobrança/dossiê **CP** aguardam consistência

Frase pronta: “todo caminho de dado tem resposta para a falha — *durabilidade, ordem, idempotência e reconciliação* sustentam ‘sem perda de pacotes’ e ‘ordenação estrita sob estresse’.”

8. Glossário essencial (domine estes termos)

- **Edge Computing:** processar no dispositivo (veículo), perto da fonte, antes de enviar.
- **Backpressure (contrapressão):** o consumidor mais lento não é atropelado; o sistema regula o fluxo.
- **Idempotência:** aplicar a operação $2\times$ dá o mesmo resultado que $1\times$ (essencial para não cobrar em dobro).
- **Store-and-forward:** guardar localmente na desconexão e reenviar depois, sem perder dados.
- **Log-Structured Merge (LSM):** estrutura de escrita rápida do Cassandra (memtable → SSTable → compaction).
- **Consistent hashing:** distribuir chaves num anel para balancear e reequilibrar com pouca movimentação.
- **Quórum N/R/W:** N réplicas, leitura exige R, escrita exige W; se $R+W>N \rightarrow$ consistência forte.
- **Vector clock:** relógio lógico para detectar causalidade/conflitos entre réplicas.
- **Watermark:** marca “já vi tudo até o tempo T” para fechar janelas com dados fora de ordem.
- **Time travel:** consultar uma versão passada da tabela (Delta) — auditoria e dossiês reproduzíveis.
- **Data skipping:** pular arquivos irrelevantes usando estatísticas min/max (acelera consultas).
- **Embedding:** vetor que representa o significado de um texto/dado, usado em busca por similaridade.
- **HNSW:** algoritmo de busca aproximada de vizinhos (ANN) usado por bancos vetoriais.
- **Medallion (Bronze/Silver/Gold):** camadas de refino de dados no Lakehouse.
- **Fact constellation:** vários fatos compartilhando dimensões conformadas.
- **SCD Tipo 2:** versiona mudanças de dimensão (guarda histórico com vigência).

9. Cheat-sheet das 21 referências (uma linha cada)

#	Referência	Ideia-chave (o que citar)
1	Ghemawat 2003 (GFS)	Falha é a norma; append-only + checksum
2	Dean & Ghemawat 2004 (MapReduce)	Map/reduce paraleliza e tolera falha; localidade; combiner
3	Chang 2006 (Bigtable)	Wide-column esparso sobre GFS; composição em camadas
4	Zaharia 2012 (RDD/Spark)	Tolerância por linhagem; memória; DAG
5	Armbrust 2015 (Spark SQL)	DataFrame + otimizador Catalyst
6	Zaharia 2013 (D-Streams)	Micro-batch com exactly-once idempotente
7	Carbone 2015 (Flink + ABS)	Dataflow único; event-time/watermark; snapshots sem parar
8	Kreps 2011 (Kafka)	Log particionado; ordem por partição; replay por offset
9	Fragkoulis 2020 (survey stream)	Out-of-order + watermarks; 2ª geração; $Kappa > Lambda$
10	Armbrust 2020 (Delta Lake)	Log de transações $\rightarrow$ ACID/time travel no object store
11	Armbrust 2021 (Lakehouse)	Unifica lake + DW; acaba com staleness e TCO

#	Referência	Ideia-chave (o que citar)
12	Chaudhuri & Dayal 1997 (DW/OLAP)	OLAP separado do OLTP; star schema; roll-up/drill
13	Gray 1997 (Data Cube)	Operador CUBE + ALL; distributivo/algébrico/holístico
14	Corbett 2012 (Spanner)	TrueTime; consistência externa; NewSQL
15	DeCandia 2007 (Dynamo)	Always-writeable; quórum N/R/W; percentil 99,9
16	Lakshman 2009 (Cassandra)	LSM; clustering por timestamp = ordem física
17	Gilbert & Lynch 2002 (CAP)	Sob partição, escolha C ou A
18	Cattell 2011 (NoSQL survey)	4 modelos; BASE vs ACID; poliglota
19	Lewis 2020 (RAG)	Recuperação reduz alucinação; proveniência; hot-swap
20	Guo 2022 (Manu, vetorial)	Banco vetorial; log-as-data; memória de agente
21	(Edge/IoT — conceito)	Filtrar na borda; banda como recurso escasso

10. Flashcards / treino de Q&A (cubra a resposta e teste-se)

- P: Por que Kappa e não Lambda?** R: Problema é streaming-first; Lambda mantém dois códigos e arrisca divergência (Fragkoulis 2020). O Kafka durável unifica; o lote pesado é coberto pelo Lakehouse/Delta.
- P: Como garantem a ordenação estrita por veículo?** R: Partição do Kafka por `vehicle_id` (ordem na partição — Kreps 2011) + event-time com watermarks no Flink (Carbone 2015); timestamp carimbado na borda; objetivo é a consistência externa do Spanner (Corbett 2012).
- P: Exactly-once é real?** R: Ingestão = at-least-once + chave de idempotência para dedup; exactly-once de fato nas etapas internas (D-Streams idempotentes; ABS no Flink).
- P: Como não perder pacotes com rede oscilante?** R: Kafka é log durável com replay; a borda faz store-and-forward; o modelo always-writeable do Dynamo inspira a camada de telemetria (Cassandra).
- P: Onde cada banco cai no CAP?** R: Telemetria = AP (disponível, eventual); cobrança/dossiê = CP (consistência forte). Segmentar por trade-off (Gilbert & Lynch 2002).
- P: Por que Cassandra e não só Mongo?** R: Cassandra é write-optimized (LSM) e faz clustering por timestamp na partição `vehicle_id` — materializa a ordem (Lakshman 2009); Mongo é para cadastrais/dossiês.
- P: O que é o Lakehouse e por que Delta?** R: Unifica lake + DW; o log de transações do Delta traz ACID e time travel ao object store (Armbrust 2020/21), acabando com staleness e dois códigos.
- P: Diferença ETL x ELT?** R: ELT carrega o bruto e transforma no cluster (Silver→Gold); mais escalável e é o que a ementa moderna pede.
- P: Como o DW da Aval. 02 se conecta?** R: Vira a **camada Gold** do Lakehouse, estendida com 4 fatos e novas dimensões carregados por ELT do Silver.
- P: Onde aparece OLAP/CUBE?** R: No dashboard: roll-up frota→pátio→veículo e cubo N-dimensional com ALL (Gray 1997), sempre sobre a Gold (Chaudhuri 1997).
- P: O que é RAG e por que usar?** R: O gerador consulta um índice denso antes de responder → menos alucinação, proveniência, troca de índice sem re-treino (Lewis 2020). Base do concierge e dos dossiês.
- P: Banco vetorial — para quê?** R: Memória (embeddings) de LLM/agente, busca por similaridade/HNSW (Guo 2022); ex.: Milvus/Weaviate/pgvector.
- P: NewSQL — onde entra?** R: SQL+ACID que escala (Spanner — Corbett 2012); aqui o PostgreSQL é o polo consistente da cobrança e da modelagem dimensional.
- P: MapReduce ainda importa?** R: Como paradigma (map/reduce → paralelismo e tolerância — Dean 2004); o Spark o generaliza (DAG, in-memory, linhagem — Zaharia 2012); por isso descartamos o Hadoop clássico.
- P: Por que filtrar na borda?** R: Banda é o recurso escasso (localidade — Dean 2004); vídeo bruto fica no veículo, só agregados e exceções sobem → ↓ custo e ↓ latência crítica.

16. **P: Como o Flink lida com dados fora de ordem?** R: Event-time + watermarks reordenam pelo timestamp de origem; ABS dá snapshots consistentes sem parar (Carbone 2015).
 17. **P: Como garantem ACID entre batch e streaming?** R: Delta Lake — ambos operam sobre as mesmas tabelas transacionais; time travel para auditoria.
 18. **P: Orquestração dos pipelines?** R: Airflow (DAG) orquestra o ELT Silver→Gold no perfil full (ementa cita Airflow/Prefect/Dagster).
 19. **P: Grafo (Neo4j) — para quê?** R: Roteirização e raciocínio do agente (relacionamentos); perfil full.
 20. **P: Como validaram sem rodar tudo?** R: docker compose config valida os perfis; 115 testes unitários (combiner, cobrança exactly-once, Markov=1.0, SCD-2, RAG); DDL 01+06 roda limpo num PostgreSQL real.
 21. **P: Qual o maior custo e como controlam?** R: Banda/egress → filtrar na borda; compute com autoscaling+spot; storage com tiering e TTL.
 22. **P: Privacidade/LGPD?** R: Consentimento p/ agenda, PII, wake-word on-device; governança e time travel do Delta para auditoria.
-

11. Autoavaliação (responda sem olhar — teste-se)

1. Explique, sem consultar, o pipeline completo do dado (do veículo ao dashboard) nomeando cada tecnologia.
 2. Diga uma referência para cada uma das 4 partes da ementa e a ideia que ela sustenta no projeto.
 3. Desenhe o esquema estrela original (3 fatos, 5 dimensões) e aponte a dimensão role-playing.
 4. Justifique Kappa vs Lambda em 3 frases.
 5. Para telemetria, cobrança e cache, diga o banco, o modelo NoSQL e o lado do CAP.
 6. O que é watermark e que problema resolve? Dê o exemplo do túnel.
 7. Como o Delta Lake dá ACID a um object store que não tem transações?
 8. O que a extensão do DW acrescentou e por quê (cite 2 fatos novos e 1 dimensão SCD-2)?
 9. O que é RAG e por que ele aparece tanto na cobrança/dossiê quanto no concierge?
 10. Diferencie agregado distributivo, algébrico e holístico e diga o impacto na divisão hot/batch.
-

12. Pegadinhas comuns da banca (evite estes deslizes)

- **Não diga** que o Spark “é” MapReduce — diga que ele **generaliza** o modelo (DAG, memória, linhagem) e supera o Hadoop clássico.
 - **Não diga** “exactly-once” como garantia absoluta desde a borda — é **at-least-once + dedup por chave**; exactly-once é das etapas internas.
 - **Não chame** “Avro/JSON” de “binário compacto” — só o **Avro** é binário; JSON é texto verboso (o que se evita).
 - **Não rode** o dashboard OLAP sobre o OLTP — é sobre a **Gold** (axioma de Chaudhuri & Dayal).
 - **Não confunda** processing-time com **event-time** — a ordenação é pelo timestamp do **veículo**.
 - **Não trate** o Kafka como armazenamento histórico eterno — retenção é finita; o histórico longo é o **Delta Bronze**.
 - **Não esqueça** de amarrar cada decisão técnica a um **valor de negócio** — é o que separa uma boa defesa de uma ótima.
-

Bons estudos. Se dominar §1 (mapa mental) + §10 (flashcards), você defende com segurança; §2–6 são o aprofundamento para as perguntas difíceis.